



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala

Universidad Rafael Landívar

Facultad de ingeniería

Manejo e implementación de archivos

Sección: 03

Ing. Luis Enrique Rojas

Facultad de Ingeniería

Facultad de Informática y sistemas

Manejo e Implementación de Archivos

Miércoles, 30 de septiembre de 2026



LABORATORIO 13
API SWAGGER & POSTMAN

Shayla Mayré De León Monterroso

Carné: 1342925

Guatemala de la Asunción, 30 de septiembre de 2026



Nombre: Shayla Mayré De León Monterroso **Carné:** 1342925

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
R// Es una forma de comunicación entre aplicaciones por medio de internet. Permite que un programa solicite, envíe, modifique o elimine información utilizando métodos HTTP como GET, POST, PUT y DELETE.
2. ¿Qué es un endpoint?
R// Es una dirección específica dentro de una API que permite acceder a un recurso o realizar una operación determinada.
3. ¿Qué diferencia existe entre una URL y un endpoint?
R// Una URL es una dirección utilizada para localizar un recurso en internet. Un endpoint es una URL específica de una API que está preparada para recibir una solicitud y ejecutar una operación.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
R// Get: consulta u obtiene información, Post: crea un nuevo recurso, Patch: modifica solamente algunos datos de un recurso, Delete: elimina un recurso.
5. ¿Qué es JSON y por qué es común en las APIs?
R// Es un formato utilizado para representar e intercambiar información mediante pares de clave y valor. Es común en las APIs porque es sencillo de leer y puede ser interpretado fácilmente por diferentes lenguajes de programación.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
R// Un código de estado HTTP indica el resultado de una solicitud realizada a un servidor.
200: la solicitud se realizó correctamente
404: el recurso solicitado no fue encontrado
500: ocurrió un error interno en el servidor
7. ¿Qué información se puede encontrar en la respuesta de una API?
R// Puede incluir un código HTTP, encabezados y un cuerpo con información. Normalmente el cuerpo contiene los datos solicitados en formato JSON.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
R// Postman permite enviar solicitudes directamente a una API sin necesidad de crear primero un programa. Esto facilita verificar que los endpoints, parámetros, cuerpos JSON y respuestas funcionen correctamente antes de integrarlos en una aplicación.
9. ¿Qué es y para qué sirve Swagger UI?
R// Es una herramienta que muestra de forma gráfica la documentación de una API. Permite conocer sus endpoints, métodos HTTP, parámetros y estructuras de datos.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
R// Postman permite crear, organizar y ejecutar solicitudes HTTP hacia una API. Puede utilizarse para probar operaciones como GET, POST, PUT y DELETE, analizar las respuestas y verificar si la API funciona correctamente.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

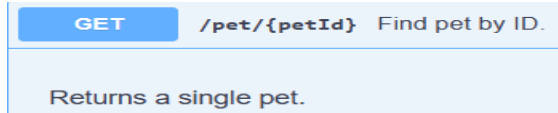
Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.



2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.



- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA -



- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

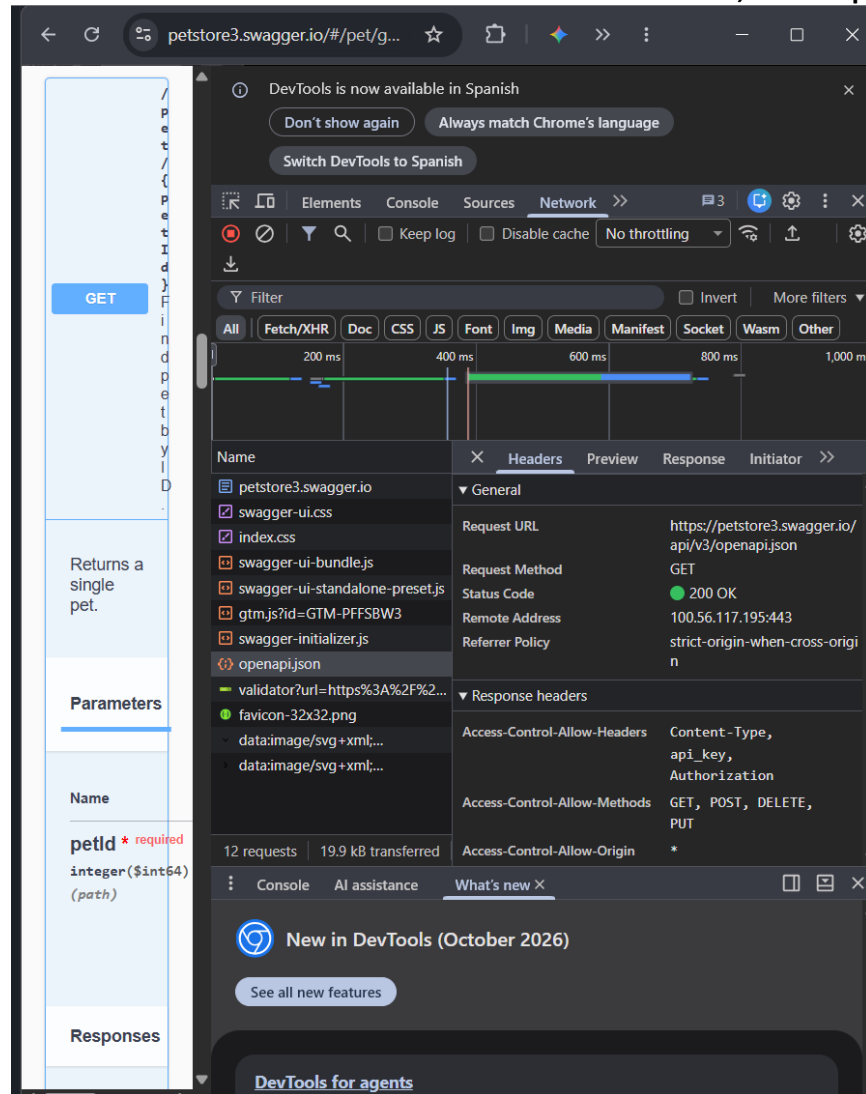
Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.